A ledger for Lichen

validity, finality and exit when the hub is the sequencer

draft

waxwing

Version 0.1, October 2026

a companion to *Lichen*, version 0.3 [1]

Abstract

Lichen [1] lets a user withdraw a hub's L2 tokens as bitcoin without trusting the hub, given a proof that her withdrawal is correct by the L2's own account of its ledger. Like the BitVM bridges, the main paper leaves that account to the L2. This note describes lichen's own L2, in which the hub is also the sequencer, so that the party that pays out controls the ledger it is judged by. That merger forces the withdrawal statement to enforce what a separate L2 would vouch for: that every block was valid, that there is one history and that it is final on Bitcoin, and that a user can leave from a state already anchored even if the operator censors her or vanishes. We describe the ledger, a chain of validity proofs, anchoring by a chain of single-use seals with the contract's work on top, and the exit: Alice's claim from an anchored state, the hub's counter to it, and the statement's three branches, all riding on the same BABE gadget as the main paper's. All of it is built. The cooperative path is unchanged. An exit's dispute is about 15 kvB, against 9.7 for a refused withdrawal, and the hub's proof of a counter takes minutes. The worst case falls on Alice: proving a forged counter false verifies a Groth16 proof inside RISC Zero, 56.3M cycles, about an hour on a laptop. It needs none of her secrets and has days to run, so it can be delegated, but it is a real burden on an ordinary user, and we set out the alternatives. Data availability remains the main open problem.

Contents

1 Scope **2**

2 The ledger **3**

3 What the merged operator can do **4**

4 Validity **4**

5 Finality **5**

6 The withdrawal statement **5**

7 The exit **6**

7.1 The claim . . . 6
7.2 The counter . . . 7
7.3 On the gadget . . . 7

8 What the exit costs **8**

9 Alice's worst case **9**

10 Open questions **11**
10.1 Data availability . . . 11
10.2 The rest . . . 11

1. Scope

The main paper [1] describes a contract between a user, Alice, and a hub: a 2-of-2 output holding her deposit, the hub's liquidity up to a cap $V_{\max}$, and two reserves. The hub's tokens on an L2 are collateralised by it; a withdrawal of b is a return of b tokens to the hub with the contract's memo c, folded cooperatively or, if the hub refuses or is offline, enforced on Bitcoin by a verification gadget, BABE [5] with the amount authenticated by the hub's one-time signature [2]. What the gadget enforces is a statement fixed when the contract opens. At its simplest that statement is:

> *a return of b to contract c exists, by the L2's account of its ledger.*

How the L2 makes its account trustworthy is its own business, as it is for the BitVM bridges [6]: an L2 secured by its own consensus, or a rollup on another chain with its own proofs, supplies the account and lichen consumes it. The hub is then only a counterparty; it cannot rewrite a ledger it does not run.

This note describes the L2 that lichen builds [3]. In it the hub and the sequencer are one operator, or two that trust each other. That is a design choice, and its consequence is that the L2's account is the hub's own word. A hub that profits from defaults, since whatever isn't withdrawn by T_{close} is its, has every reason to bend that account. So everything a separate L2 would vouch for has to be enforced inside the withdrawal statement, against the operator. We build it in three levels:

- P_0, the main paper's: the account is trusted.
- P_1, *validity and finality*: the return is in a state that ends a valid history from genesis, that history is the one anchored on Bitcoin, and the anchor has the contract's work on top (Sections 4–6).
- P_2, *the exit*: Alice can leave from a state already anchored, without a return, against an operator that censors her return, stops sealing or vanishes; the hub may answer only with a counter showing that she spent what she claims (Sections 7–9).

The gadget does not change between the levels; the statement does, and from P_2 on, so do the first two transactions of the dispute, which carry the claim and the counter. Section 10 lists what is left, data availability first.

Figures marked (lichen) are measured in lichen's proof of concept, on regtest under mainnet's

relay policy or on Mutinynet as stated; the rest are estimates, marked (est.). Proving times are on one 14-core Apple M4 Pro laptop, on its CPU (the RISC Zero version we use has no Metal prover), or on a rented 4-vCPU x86 server, as stated.

2. The ledger

The ledger is one of notes. A note is an owner, a value, a memo and a random ρ; its commitment is a SHA-256 hash of these, and the commitments form an append-only Merkle tree, so a note's path under a root stays valid however the ledger grows. A spent note is not removed but recorded by its *nullifier* in a second tree, a sparse Merkle tree keyed by nullifier, whose leaf records how the note was spent:

- *spent*: an ordinary transaction;
- *returned*(c, r): a transaction paying the hub exactly one note, of r with memo $c \neq 0$, the first step of a withdrawal from contract c;
- *exited*(c): contract c's exit record (Section 7).

The tags matter only to the exit, whose counter must tell which spends may be held against which claim. A state root SR hashes the height, the commitment tree's root and the nullifier tree's root, and the sequencer signs it (BIP340).

There are two kinds of note in one tree. *Transparent* notes have a public owner key and value: the hub's mints (a deposit's tokens) and returns to the hub are transparent, and a transparent spend is authorised by the owner's signature. *Shielded* notes publish only their commitment; the owner's authority is knowledge of a key nk, the nullifier is derived from it so that nobody can link it to its note, and the sender proves the spend in zero knowledge on her own machine [4]. The transaction types are a mint, a transfer, shielding and unshielding, a return, an anchor record (Section 5) and an exit record (Section 7).

Three proofs, all RISC Zero, are involved (Table 1).

proof	made by, when	shows; cost
spend	the sender, for each shielded transaction	its notes exist, are hers and are unspent, and the amounts balance; 6–30 s on an 8-core Mac (lichen)
chain	the sequencer's prover, for each sealed height	this block is a valid transition, and so was the history before it (each proof assumes the last); about 40 s for a small height, 142k cycles, on the laptop in Docker (lichen)
withdrawal	the user, or anyone she delegates to, only in a dispute	her statement, P_1 or P_2 (Sections 6 and 7); wrapped in Groth16 for the gadget

Table 1: The three proofs.

3. What the merged operator can do

With the hub and the sequencer one party, the threat is the operator against its users. Users against users is the ledger's ordinary job, and the operator against itself is out of scope. Table 2 lists what the operator could do with an unchecked account, and what answers each. A vanished operator needs no new mechanism: a hub that never responds to a `request` lets

the operator	answered by
mints notes nobody paid for, and withdraws them from users' contracts	validity: the chain proof (Section 4)
signs two valid histories that split, paying Bob in one and returning the same money to the hub in the other	one anchored history: the seal chain (Section 5)
anchors a history on a branch of Bitcoin it mined	the contract's work W on the anchor (Section 5)
censors Alice's return or stops sealing, so that she has nothing to prove, and takes her contract when she can't	the exit (Section 7)
withholds the data of something it anchored	not yet answered: data availability (Section 10.1)

Table 2: What a merged operator could do, and what answers it.

`request_timeout` pay Alice after its window, whatever she asserted [2]. The exit is for one that is present but hostile: it responds, then censors her return or stops producing the chain receipts she needs, so that she can't prove by the deadline and the contract is taken from her.

4. Validity

The chain guest proves one sealed height: the block's transactions applied to the previous state give the new one, its spends authorised (a signature, or a verified spend proof for a shielded note), no nullifier twice, values balanced, mints signed by the hub. It takes the previous height's chain receipt as an assumption, so each receipt vouches for the whole history from genesis. Its journal carries the height, the state root, a window of recent roots, the anchoring state of Section 5, and a history hash

$$hist_0 = 0, \qquad hist_h = H(\texttt{"lichen/hist"} \,\|\, hist_{h-1} \,\|\, SR_h),$$

which commits the whole sequence of roots to h, so that an earlier state's place in a later one's history is a hash-forward check.

Costs (lichen): a block of one mint is 395,543 cycles, a block holding only an anchor record about 130,000, both one segment; on the laptop a small height proves in about 40 s. The prover runs behind the seals, so for an honest sequencer the lag is only a delay, but every withdrawal waits for the chain receipt at its return's height. One machine proving height after height keeps up with on the order of a hundred transactions an hour; beyond that the remedy is the operator's: a GPU, or proving blocks in parallel and chaining them afterwards.

A valid history is not a unique one. The sequencer can sign two histories that split at a height, each valid, which the chain proof cannot see. The wallet and the hub record every root they are shown and refuse to go on when shown a second root at a height, with the evidence; that makes equivocation visible to whoever looks, and anchoring makes it useless.

5. Finality

The seal chain. The deployment's genesis names an outpoint, the first *seal*, a P2TR output of the sequencer's key. An anchor spends the current seal and creates the next, beside an output `OP_RETURN "lichen"` $\| \; h \; \| \; hist_h$, committing the history to a sealed height h; it is 269 vB (lichen). Bitcoin lets an output be spent once, so there is one chain of anchors, after Todd's single-use seals [7]. The sequencer anchors at most once a Bitcoin block, and only when something has changed.

Anchors inside the history. A linear chain of seals could still commit roots of two histories in turn. So once an anchor confirms, the sequencer records it on the L2, and the chain guest checks the record: the transaction is in the block it names, by Merkle path to the header; it spends the seal the journal names, whose script is checked so that no one else could spend it and stall the anchoring; and it commits a *hist* that is an ancestor of the current one. A history forked from the anchored one can then hold no anchor after the fork: the only spend of the seal commits the other history.

The contract's work. The chain guest checks no proof of work: an anchor on a branch of Bitcoin costs the work on top of it, whatever the guest checked. The withdrawal statement checks it instead: headers that follow the anchor's block, each linked and within its own target, whose work sums to at least W. W is fixed when the contract opens, ten blocks' work at the difficulty of that moment (its checkpoint, which both parties check against their own view of the chain), and it goes into the contract's identifier,

$$\mathcal{D} = H(\texttt{"lichen/contract"} \,\|\, \text{deployment} \,\|\, \text{genesis seal} \,\|\, W),$$

the deployment being the sequencer's and the hub's keys. A forger gains nothing from headers declaring low difficulty: work is summed, so headers worth W cost about W hashes however it is split. About ten headers cost the statement a few thousand cycles. This is not a proof of the heaviest chain; it makes a forgery a matter of mining a fork of Bitcoin as long as W, which bounds the hub's exposure to one forged history by the reward of the blocks a fork replaces.

For receivers. A payment is final once its height is anchored ten blocks deep. The wallet does not take the sequencer's word for the anchors: it computes *hist* from the roots it has checked and finds each anchor on Bitcoin itself, so its balance splits into final, anchored and sealed only.

6. The withdrawal statement

At P_1 a withdrawal proves:

1. a note of b, owned by the hub with memo c, marked a return, in the commitment tree of a state root SR, by Merkle path;

2. SR is the sequencer's, by its signature at its height;

3. a chain receipt, at that height or any later one, whose anchoring state commits SR: validity from genesis, in the one anchored history;

4. headers on the anchor's block with work at least W.

Under BABE the statement also checks the hub's one-time signature on b, and its journal is $c \,\|\, \mathcal{D} \,\|\, H(pk_H)$, the same for every amount [2]. The chain guest's image id is built into the withdrawal guest, so the one image id the contract pins covers both. The statement is 389,020 cycles, 482k with the hub's signature, one segment either way (lichen); it proves in 74 s on the laptop and in 640–660 s on the server, and wraps in about 4 minutes.

The cooperative path. The hub checks the same statement natively, in milliseconds, against the chain receipt and its own node's headers, and folds: one transaction of 193 vB. So it pays out only what a dispute could win, and Alice proves nothing. Anchoring adds a wait, not a cost: the next anchor and W on it, about two hours on mainnet.

7. The exit

A withdrawal starts with a return, which the sequencer must include. A merged operator profits from not including it. The exit lets Alice leave without one:

> *these notes of mine were unspent at the anchored state S; pay me b.*

The operator may answer once, only with a *counter*: something only Alice could have done, showing she no longer holds what she claims. It can't stop an exit otherwise: a censoring operator has no counter to a true claim, and a vanished one times out.

7.1. The claim

Alice claims from her *exit kit*, which the sequencer serves after each of her spends and receipts and her wallet checks against the signed root: at an anchored state S, each note's path in the commitment tree and its nullifier's absence from the nullifier tree. A kit never goes stale: the tree only grows, and absence from S's set stays true of S. If the operator stops serving, her last kit still works: if she hasn't spent since, no counter exists. A payment is final for its receiver once the receiver holds its kit.

The claim's fingerprint is

$$E = H(c \,\|\, O_D \,\|\, b \,\|\, h_S \,\|\, SR_S \,\|\, \mathit{nullifiers}),$$

O_D being the contract's deposit coin, which only its funding can spend, so that it names the contract on Bitcoin and is known when the setup fixes the statement. Each claimed note's owner signs E, and `request` posts the claim: h_S, SR_S, each note's opening and its owner's signature, at most 1,169 bytes for eight notes, in fifteen witness items of up to 80 bytes, since Bitcoin Core relays no larger tapscript witness item. The statements read it through the coinbase's witness commitment. It is posted so that the hub knows which nullifiers to search for, and so that the signatures can be cited against her. A claim spends whole notes and the contract pays once, so the wallet claims everything it holds, up to $V_{\max}$; value above b in the claimed notes is forfeit, and the wallet sizes its notes to fit.

7.2. The counter

A counter shows one of the claimed notes gone, on one of two grounds.

A spend. The note's nullifier is in the nullifier tree of an anchored state S' after S, in the same history (S''s chain receipt hashes forward through S), with a leaf that counts:

- *spent* counts if S''s anchor confirmed in or before `request`'s block. A later spend could be a transaction of hers that the operator held and sealed after she committed to her claim; the cutoff leaves only what was final on Bitcoin when she claimed. `request` must have the contract's work on it too, or either side could forge the cutoff with a cheap header;
- *returned*(c', r) counts the same way, unless $c' = c$ and $r \geq b$: then the hub holds at least b of her tokens for this contract, which pays once. A smaller return, or one to another contract, can't shelter a spend;
- *exited*(c') counts whenever anchored, unless $c' = c$: a record of the same notes claimed through another contract.

An equivocation. The owner of one of the notes signed a claim of it through another contract. Under BABE the hub responds before Alice proves, so two requests through two contracts in one block would both fall due before either exit record existed; her signature on the other claim is the evidence instead. Only the owner can sign one, a copied signature names its own contract, and an honest wallet never signs a claim of a note through two contracts. No L2 state is involved.

Why the operator can't forge one. A nullifier enters the set only by a spend, which needs the note's keys, or by an *exit record*, which the hub posts once a `request` confirms, to retire the claimed notes, and which the chain guest accepts only with the owners' signatures on that claim. So every counter traces to something the owner did. Her own return to this contract never counts, so a return she signed, which the operator still holds, can't be sealed after her claim and used against her.

7.3. On the gadget

The claim and the counter ride on the dispute's first two transactions; the rest of the graph is the main paper's [2].

- `request` signs, beside b, the claim's E under a second one-time key of Alice's, and posts the claim. A return's dispute signs a fixed E and posts nothing, so one graph serves both.
- `respond` has two leaves, pre-signed at opening, both comparing the hub's b and E with Alice's digit by digit: *plain*, the hub signs E under a key e_{plain}; *countering*, it signs E under e_{counter} and the counter K itself.

K is a Groth16 proof of the counter guest, whose journal is $\mathcal{D} \,\|\, E$, compressed to 128 bytes and signed whole, in 259 digits. It is signed rather than hashed because no script can tie a hash of posted bytes to signed digits without `OP_CAT`: a hub that signed $H(K)$ could post other bytes and leave Alice nothing to prove false. Uncompressed, at 256 bytes, it would exceed the 1,000-element stack limit at two elements a digit.

The statement's three branches. Alice's statement is fixed at setup, with the journal $c \,\|\, \mathcal{D} \,\|\, H(pk_H \,\|\, O_D)$, pk_H the hub's four one-time keys. As b does, the hub's E, its mode and K enter as private inputs, each authenticated by the hub's one-time signature under a key the statement fixes, and Script checks the same signatures in `respond`. Alice wins on any of:

- *return*: the hub signed b, and a return of b to c is anchored (P_1), in either mode;
- *claim*: the hub signed b and E in the plain leaf, and her claim holds: her notes in S's tree and unspent there, signed by their owners, distinct, S anchored with W on it, `request` in a block with W on it too and posting exactly this claim, b at most their sum;
- *failed counter*: the hub signed E and K in the countering leaf, and K is not a valid counter for her $\mathcal{D} \,\|\, E$.

Which key signed E says which leaf ran. A true counter leaves her no branch, and `respond_timeout` takes the contract once the deadline T_A passes. A countered claim loses the whole contract, as any false claim does; a careful wallet never makes one. A failed counter pays Alice whatever her own claim: a counter is always the hub's choice, and one that fails is one it chose to forge.

The hub's side. When a claim's `request` confirms, the hub reads the claim and searches: another of its contracts' requests signed by one of the owners, another contract's exit record, or a spend anchored by `request`'s block, its own return of at least b excluded. With a ground, it assembles the counter guest's input, waits until the blocks it needs are deep enough, proves and wraps the counter, and responds countering. With none, it asks the sequencer to refuse spends of the claimed nullifiers, records the exit on the L2 and responds plainly. An honest hub may fold an exit instead, having checked it natively.

What it guarantees. Not that every redemption consumes exactly one L2 entitlement whatever happens: a dead operator posts no exit records, so a note can be redeemed twice. The invariant is instead that no user loses to another or to the operator, since an honest claim can't be countered and every payment received stays good; that every payout not matched by value retired on the L2 is the operator's loss; and that an operator that stays live and follows the protocol can avoid that loss. The last fails within the anchor's latency until sender acknowledgment (Section 10.1): a dishonest Alice who pays Bob and requests before the payment's anchor confirms is paid, and Bob keeps the payment, at the hub's cost, once per contract.

8. What the exit costs

The proofs. Table 3 gives each statement's cycles, run on made-up inputs with the chain journal assumed (lichen), and its proving time. Given a valid counter, Alice's statement refuses her, both natively and in the guest (lichen). The hub's proof is a small part of its window. The failed counter is Alice's worst case, the subject of Section 9.

On chain. On regtest under mainnet's relay policy (lichen): `request` 2.37 kvB with the largest claim, against 0.45 kvB without the exit; `respond` 4.48 kvB plain and 12.26 kvB countering (766 witness elements), against 0.76 kvB. So a refused true claim is about 15 kvB rather than 9.7,

statement	cycles, segments	proving
a return's dispute, with the exit's guest	489k, 1	640 s and a 257 s wrap on the server (lichen)
a claim of one note	1.74M, 2	122 s on the laptop, lifted and joined (lichen)
a claim of eight notes	5.27M, 6	about 6 min on the laptop (est.)
the hub's counter, a spend	594k, 1	about a minute on the laptop (est.)
the hub's counter, an equivocation	309k, 1	30 s on the laptop; 306 s and a 252 s wrap on the server (lichen)
a failed counter	56.3M, 58	about an hour on the laptop, and a 134 s wrap; about ten hours on the server (est., from measured segments)

Table 3: The exit's statements: cycles, and proving times.

and an exit from a vanished hub, `request` and its timeout, about 2.9 kvB rather than 1.2 (est., from these sizes). None of it grows with misbehaviour.

Windows. The hub must find and prove a counter within its response window w_H: the contract's work on `request`'s block first, then the proof and the wrap. So w_H is 240 blocks on Mutinynet (two hours), 36 on signet (six hours) and 72 on testnet4. Alice's proof has no window: `assert` is due by an absolute deadline T_A, shortly before T_{close}.

Congestion. The main paper's rule, $Ns \leq w \cdot 1\,\text{MvB}$ for every window, applies to the hub's moves. Its windowed sum grows from 21.5 to about 25.2 kvB, or 33 kvB if every response counters, so a dispute of a week holds about 40,000 contracts (30,500) rather than 47,000 (est.) [2]. Alice's windowed move is unchanged. A wave of stale claims would make the hub prove many counters in one w_H; it is costly to mount, since each countered claimant loses a contract, and counters prove in parallel.

9. Alice's worst case

Why it is heavy. The failed-counter branch must show that a Groth16 proof does *not* verify. RISC Zero composes receipts cheaply, as the withdrawal does the chain receipt, but composition only shows that a receipt does verify; showing that one fails means running the verifier. K has to be a Groth16 proof, not a RISC Zero receipt, because a `respond` leaf can bind only about 128 bytes of signed digits. So the guest verifies BN254 Groth16 in software, with RISC Zero's 256-bit accelerator: 56.3M cycles with RISC Zero's fork of substrate-bn, 174M with arkworks (lichen). A profile of the 56.3M (lichen): the pairings 44%, of which Miller loops 10.7M, the final exponentiation 8.2M and precomputation 6.6M; checking that the verifying key's constant G2 points are in their subgroup 31%; the proof's own points' decompression and subgroup checks about 5.5M; the public input's scalar multiplications 8%.

What it takes. The branch's segments prove in about 53 s each on the laptop and take 11 s more to lift and join, so about an hour, then 134 s to wrap. On the 4-vCPU server it would be

about ten hours. RISC Zero proves on the GPU with CUDA only; at the throughput it reports for an RTX 4090, about 1.4 MHz [9] (not checked by us), the segments take about a minute (est.).

Why it is tolerable. Three things soften it.

- *It is reached only by a forgery.* A hub that counters falsely loses the contract, so a rational hub never makes Alice run it.
- *It has days.* `assert`'s deadline is absolute, close to the contract's end; an hour, or ten, is no constraint.
- *Anyone can prove it.* Its inputs are on chain (E, K and the hub's signatures) or in the journal (c and $\mathcal{D}$), so it needs no secret of Alice's: a proving service, her tower or a friend can make it, and she signs the Groth16 proof she is handed after checking it.

But a deterrent counts only if it is credible, and an ordinary user on a phone cannot prove for an hour. In practice the branch presumes proving infrastructure that someone runs for her. Every branch already asks her for a proof and a wrap, minutes on a laptop; this one asks thirty times more, and that is where the design is weakest.

Alternatives. What we know of, and where each stands:

- *A cheaper verifier.* Skipping the subgroup checks of the verifying key's constants and precomputing its constant terms would bring the branch to about 31M cycles (est.): half an hour on the laptop. A new guest, so a new deployment.
- *The equivocation ground without a SNARK.* That ground needs only the owner's signature, so the hub could sign the evidence itself instead of a proof of it. It does not touch the worst case, which the hub can always choose, but it shrinks the countering `respond` and spares the hub a proof.
- *The hub signs the counter's facts, not a proof* (which note, how it was spent, at which height and root), and Alice refutes them from her own data, cheaply. Rejected: the operator can anchor a state it never proves, with her nullifier in it, and cite it. Against a state with no valid receipt Alice has nothing positive to show, whereas K proves that a valid one exists. It would also turn every withheld later state into a weapon against idle users, whom the design now protects.
- *A second BABE instance, mirrored.* The hub proves its counter in an instance of its own, with Alice as the garbler, and E, signed by her in `request`, authenticates its input as the hub's signature authenticates b in hers. Alice's statement loses the failed-counter branch; in a dispute she checks K natively, in milliseconds, and challenges a false one. The cost moves to every contract: Alice garbles at each opening and renewal (11 s of computation on the laptop, as the hub's setup is now) and sends the hub about 240 MiB, and a contested counter costs her a `challenge_assert` of about 20.7 kvB, as the hub pays now (est., from the measured setup and sizes). With contracts of months that is a modest price, and the one alternative that removes Alice's proving altogether.

10. Open questions

10.1. Data availability

The exit is safe exactly when Alice can read the anchored data she needs. Anchors are public; their contents are not. The sharpest form is the *consolidation attack*: Alice consolidates her notes into one, as her wallet recommends; the operator anchors the consolidation and withholds the new note's data. Her old notes are countered, the consolidation really spent them, and the new one has no kit. The same works on any spend that leaves her change. If nothing has been anchored since her kit, she is safe.

A counter wins the hub a contract, so withholding is harmless whenever a spend whose data Alice can't read can't be cited. Most remedies are rules of that kind [8]:

- *Sender acknowledgment*, the planned one. A spend can be cited only with its sender's signature that she holds its data, which her wallet gives once it has her kit at the anchored state containing the spend; until then the spend's outputs are locked, and only she may claim them. No trust and nothing on Bitcoin, at the price of two-phase finality: Bob's payment is final once the acknowledgment is anchored too. It also closes the race of Section 7, since the cutoff can then allow a grace the other way.
- *Publishing each anchor's inserted nullifiers and new commitments*, about 24 vB per one-in-two-out transaction: no trust, a cost per transaction on Bitcoin.
- *A data committee in the seal*: the seal's key a threshold key of the operator and a committee, each member signing an anchor only once it holds the batch's data. One honest signer suffices; the price is the committee's liveness for progress.
- *Accept the gap*: a user is exit-safe only while the operator serves data for her latest transactions.

10.2. The rest

- The exit played on chain with real proofs, and the failed-counter branch proved whole, not only estimated from other inputs' segments.
- Alice's worst case: whether the mirrored instance, or something better, should replace the failed-counter branch (Section 9).
- Shielded notes' exits. A shielded note's authority is knowledge of nk inside the proof, with nothing to post or cite, so today a wallet exits its transparent notes only. Shielded addresses would gain a BIP340 owner key, bound into the nullifier, whose signature on E is posted like a transparent owner's; an exit would then reveal that key and link the wallet's exits, though not its spends.
- A tower for the exit. Alice's proof depends on which leaf the hub responds with, so unlike a return's she cannot sign it ahead; she must come back after `respond`, or entrust a key.
- A proof of the heaviest chain, or a counter-proof of a heavier one, in place of the work threshold W.

- Mass exit: every contract claiming by T_{close} against a hostile operator. The counter adds no games, since a failed counter pays Alice, but larger requests and a longer schedule, which favour few notes per claim, and so consolidation, which needs data availability.

References

[1] waxwing. Lichen: L2s in channels enforced by Bitcoin. Version 0.3, October 2026. https://github.com/AdamISZ/lichen/blob/master/papers/lichen-v0.3.1.pdf

[2] waxwing. Lichen over BABE: withdrawals of a variable amount, by authenticated request. Version 0.1, October 2026. https://github.com/AdamISZ/lichen/blob/master/papers/lichen-babe-v0.1.9.pdf

[3] lichen: the ledger, the sequencer, the hub and the wallet. https://github.com/AdamISZ/lichen

[4] lichen's shielded transfers: an example L2. https://github.com/AdamISZ/lichen/blob/master/docs/SHIELDED.md

[5] S. Garg, D. Kolonelos, M. Sergeevitch, S. Sridhar and D. Tse. BABE: verifying proofs on Bitcoin made 1000x cheaper. ePrint 2026/065, revised October 2026. https://eprint.iacr.org/2026/065

[6] R. Linus Woll, L. Aumayr, Z. Avarikioti, M. Maffei, A. Pelosi, O. S. Thyfronitis Litos, C. Stefo, D. Tse and A. Zamyatin. Bridging Bitcoin to second layers via BitVM2. USENIX Security 2026; ePrint 2025/1158. https://eprint.iacr.org/2025/1158

[7] P. Todd. Preventing consensus fraud with commitments and single-use-seals, 2016. https://petertodd.org/2016/commitments-and-single-use-seals

[8] J. Light. Validity rollups on Bitcoin, Appendices C and D, 2022. https://bitcoinrollups.org

[9] RISC Zero. zkVM benchmarks and datasheet. https://dev.risczero.com/api/zkvm/benchmarks